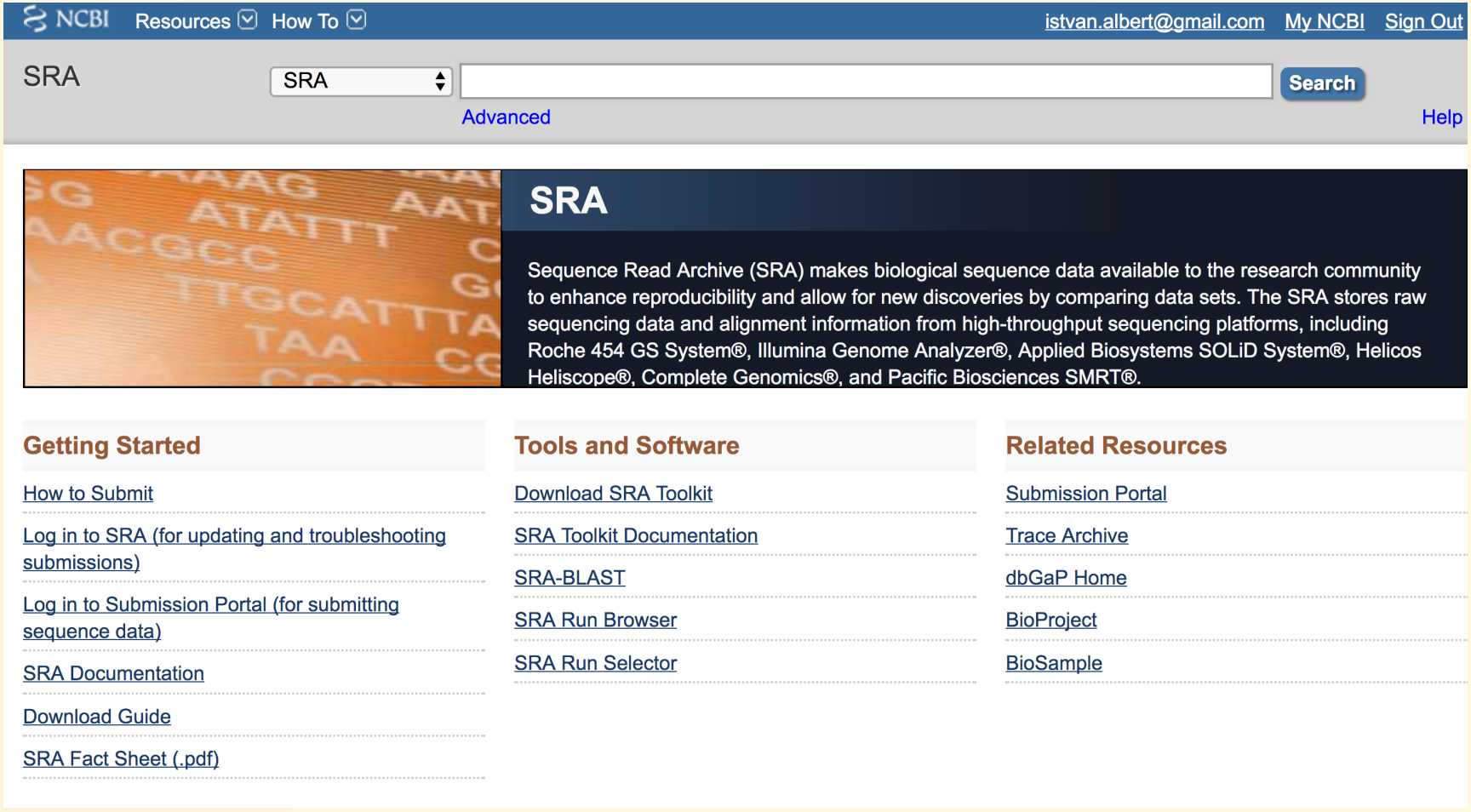
Lecture 12

Accessing the Short Read Archive

Advanced scripting

The Short Read Archive



https://www.ncbi.nlm.nih.gov/sra

SRA Naming conventions

- **NCBI BioProject**: **PRJN...**
  The overall description of a single research initiative; multiple samples and datasets.
- **NCBI BioSample**: **SAMN...** and/or **SRS...**
  A description of biological source material; a unique specimen with a unique set of attributes.
- **SRA Experiment**: **SRX...**
  A unique sequencing library for a specific sample.
- **SRA Run**: **SRR... ERR...**
  A manifest of data files linked to a given sequencing experiment.

NCBIBioProject PRJNA257197

BioProject BioProject Advanced

Display Settings: Send to:

Zaire ebolavirus

Accession: PRJNA257197 ID: 257197

Zaire ebolavirus Genome sequencing

Zaire ebolavirus sample sequencing from the 2014 outbreak in Sierra Leone, West Africa.

Accession	PRJNA257197
Data Type	Genome sequencing
Scope	Multiisolate
Organism	**Zaire ebolavirus** [Taxonomy ID: 186538] Viruses; ssRNA viruses; ssRNA negative-strand viruses; Mononegavirales; Filoviridae; Ebolavirus; Zaire ebolavirus
Publications	Gire SK *et al.*, "Genomic surveillance elucidates Ebola virus origin and transmission during the 2014 outbreak.", *Science*, 2014 Aug 28;345(6202):1369-72
Submission	Registration date: 31-Jul-2014 **Broad Institute**
Relevance	Medical

Details on BioProject PRJNA257197

Project Data:

Resource Name	Number of Links
SEQUENCE DATA	
Nucleotide (Genomic RNA)	249
SRA Experiments	891
Protein Sequences	2240
PUBLICATIONS	
PubMed	1
PMC	1
OTHER DATASETS	
BioSample	716

Each experiment will have **SRX** and **SRR** numbers.
Each sample will have **SAMN** numbers.

The SRA toolkit

The `sra-toolkit` automates access to the SRA archive.

If you knew the accession number the `fastq-dump` command can download the data for it:

```
fastq-dump -X 15000 --split-files SRR5119926
```

The `-X` will extract only a subset (1500 reads).

Note: Some datasets may be huge. Alas `fastq-dump` will first download the entire data even if you only need a small section of it ... sigh!

Windows Bash problems

`fastq-dump` has problems on Windows Bash.

Use a replacement script that we wrote called `wonderdump`. Use it the same way:

```
fastq-dump -X 15000 --split-files SRR5119926
```

on Windows Bash make that:

```
wonderdump -X 15000 --split-files SRR5119926
```

See the **Setting up Windows 10** page in the handbook.

Other Windows Problems

Currently, two main classes of line encodings exist:

---

1. **Unix style** (line feed, `LF`, `\n`)
2. **Windows style** (carriage return + line feed, `CR+LF`, `\r\n`)

Always use Unix Line endings when scripting.

Classic Mac also had another, third kind, of line ending `\r`, thankfully since 2010 Macs use Unix line endings.

Searching Bioproject PRJNA257197

We are now searching the `sra` database

```
esearch -db sra -query PRJNA257197
```

and get:

```
<ENTREZ_DIRECT>
  <Db>sra</Db>
  <WebEnv>NCID_1_69588507_130.14.18.34_9001_1506521401_402
  <QueryKey>1</QueryKey>
  <Count>891</Count>
  <Step>1</Step>
</ENTREZ_DIRECT>
```

We note the number hidden among the noise `891`

SRA result formatting

You can get a `runinfo` or a `summary.`

```
esearch -db sra -query PRJNA257197 | efetch -format runinf
```

or:

```
esearch -db sra -query PRJNA257197 | efetch -format summar
```

We do note that each command takes a lot longer than it should.

How would I know which format a database supports?

Good question comrade! That information is buried among other very "helpful" documentation at NCBI.

Everything that we know about searching NCBI comes from examples by other people.

It is rare that documentation provided by NCBI is useful.

NCBI docs tend to be circuitous, redundant, repetitive, difficult to follow, redundant and repetitive.

Runinfo and Summary

These two usually contain information that might be sufficient to identify samples but not always.

Cut the first column by comma:

```
cat runinfo.csv | cut -f 1 -d , | head
```

produces:

```
Run
SRR1972917
SRR1972918
SRR1972919
SRR1972920
```

Explore the runinfo

1. What type of information does it contain?
2. What type of information ought to be there but is not?

```
cat runinfo.csv | cut  -f 1-100 -d , | head -1
```

prints:

```
Run,ReleaseDate,LoadDate,spots,bases,spots_with_mates,
avgLength,size_MB,AssemblyName,download_path,Experiment,
LibraryName,LibraryStrategy,LibrarySelection,LibrarySource
LibraryLayout,InsertSize,InsertDev,Platform,Model,SRAStudy
BioProject,Study_Pubmed_id,ProjectID,Sample,BioSample,Samp
...
```

The summary is an XML file

An XML data file is meant to have a standardized structure to allow automated processing.

The NCBI XML file is slightly broken, and it has a few errors in it.

It **kinda works** though - like many other NCBI provided services.

Advanced XML processing

The `xtract` tool can process XML files.

It is quirky and not well explained in the docs. It can match on patterns and turn them into tabular form

```
cat summary.xml | xtract -Pattern EXPERIMENT
                                 -element PRIMARY_ID | head -3
```

Prints:

```
SRX994253 SRP045416 SRS908478
SRX994252 SRP045416 SRS908483
SRX994251 SRP045416 SRS908484
```

To use it you have to understand what is in the XML

Obtain data for multiple SRR runs

```
cat runinfo.csv | grep SRR | cut  -f 1 -d , | head -3
```

prints:

```
SRR1972917
SRR1972918
SRR1972919
```

we can get data for each with:

```
fastq-dump -X 15000 --split-files SRR1972917
fastq-dump -X 15000 --split-files SRR1972918
fastq-dump -X 15000 --split-files SRR1972919
```

How to automate processes

Store the ids in a file:

```
cat runinfo.csv | grep SRR | cut  -f 1 -d , | head -3 > sr
```

We want to repeat the command on each, but type it only once if possible.

```
cat sra.ids | xargs -n 1 echo "Hello"
```

Appends the content of each line at the end:

```
Hello SRR1972917
Hello SRR1972918
Hello SRR1972919
```

A cool trick to writing scripts

Instead of `echo` we could write `fastq-dump` and have it execute. But you may not fully understand what takes place then. So keep the `echo`:

```
cat sra.ids | xargs -n 1
                    echo "fastq-dump -X 1000 --split-files"
```

So instead of executing the command it *shows it to you*. Prints it on the screen:

```
fastq-dump -X 1000 --split-files SRR1972917
fastq-dump -X 1000 --split-files SRR1972918
fastq-dump -X 1000 --split-files SRR1972919
```

Pipe it back into bash

The previous command prints:

```
fastq-dump -X 1000 --split-files SRR1972917
fastq-dump -X 1000 --split-files SRR1972918
fastq-dump -X 1000 --split-files SRR1972919
```

Copy the first line into the terminal to test that it works.

When you are happy with how it works pipe it right back into bash to execute the script:

```
cat sra.ids | xargs -n 1
            echo "fastq-dump -X 1000 --split-files" | bash
```

Now make it into script

Collect all steps into a single script.

```
# Obtain runinfo for the project.
esearch -db sra -query PRJNA257197 | efetch -format runinfo > runinfo.csv

# Extract and keep the first few SRR numbers.
cat runinfo.csv | grep SRR | cut  -f 1 -d , | head -3 > sra.ids

# Dowload the fastq data for each SRR number.
cat sra.ids | xargs -n 1 fastq-dump -X 1500 --split-files

# Create a fastqc report for each dataset.
fastqc *.fastq
```

To add trimmomatic, we need to specify where the file names go. Can't just go to the end.

Adding trimmomatic to the script

```
1
2  # Obtain runinfo for the project.
3  esearch -db sra -query PRJNA257197 | efetch -format runinfo > runinfo.csv
4
5  # Extract and keep the first few SRR numbers.
6  cat runinfo.csv | grep SRR | cut  -f 1 -d , | head -3 > sra.ids
7
8  # Dowload the fastq data for each SRR number.
9  cat sra.ids | xargs -n 1 fastq-dump -X 1500 --split-files
10
11 # Create a fastqc report for each dataset.
12 fastqc *.fastq
13
14 # Run trimmomatic on each of the pairs.
15 cat sra.ids | xargs -n 1 -I {} trimmomatic PE {}_1.fastq {}_2.fastq -baseout {}.fq SLIDINGWINDOW:4:30
16
17 # Create a fastqc report for each improved dataset.
18 fastqc *.fq
```

The `{}` marks the spots where the file names get substituted.

Tell `xargs` to use the `{}` symbols with `-I {}`

File overload

So how many files do you have now?

```
ls -1 | wc -l
```

It prints:

```
  75
```

We processed data from `3` runs and we barely got going and already have `75` files.

You must ORGANIZE YOUR PROJECT

How to organize your data

Separate results. Every step lives in a different "output" directory.

```
mkdir -p sra
fastq-dump -X 1000 --outdir sra --split-files  SRR1972919

mkdir qc
fastqc --outdir qc SRR1972917_1.fastq

mkdir -p reads
trimmomatic SE sra/SRR1972919_1.fastq reads/SRR1972919.fq
```

Use this approach in general otherwise, you will *drown in files*.

How to split file names in bash?

If you organize your files in directories, you often run into the issue of how to split file names and find out what the directory or file names are.

For example, if you have `/data/genome.fasta.tar.gz` you might need to access:

- the directory path `data`
- the file name: `genome.fasta.tar.gz`,
- the file name without extension `genome`,
- the extension `gz`

Bash operators

I always forget which one is which. You can look them up here or Handbook (Advanced Scripting)

```
# Suppose this is the full path.
FULL=/data/foo/genome.fasta.tar.gz

# Makes: genome.fasta.tar.gz
echo $(basename ${FULL})

# Makes: fasta.tar.gz
echo ${FULL#*.}

# Only the extension: gz
echo ${FULL##*.}

# Other side: /data/foo/genome.fasta.tar
echo ${FULL#*.}
```